

BAHRIA UNIVERSITY, ISLAMABAD CAMPUS
DEPARTMENT OF COMPUTER SCIENCE

SENTINEL-IoT

An Autonomous Explainable XDR Framework for Industrial IoT
Security and Compliance Auditing

FRONTEND ARCHITECTURE & MASTER TECHNICAL SPECIFICATION MANUAL

Version 2.4 — Enterprise Production Edition — FYP-II

Lead Frontend Engineer:
Abdullah Bin Zubair Hashmi
Enrollment: 01-135231-001

Lead AI & Systems Engineer:
Wajahat Ali Tariq
Enrollment: 01-135231-091

Project Supervisor:
Dr. Muhammad Usman Hashmi
Associate Professor
Department of Computer Science

September 2026

Executive Technical Abstract

This specification serves as the comprehensive software architecture, user interface design, and communication protocol standard for the frontend implementation of **Sentinel-IoT**. Operating as an Extended Detection and Response (XDR) management console, the interface bridges a dual-head Google Conformer deep learning backbone with enterprise Security Operations Center (SOC) operations. The document formulates the technical specifications across six specialized consoles: Executive Overview, Forensic Threat Lab, Sub-Millisecond Explainable AI (XAI), Adversarial Evasion Robustness (FGSM/PGD), Autonomous Remediation with IEC 62443 Four-Eyes Safety Interlocks, NIST SP 800-53 Rev. 5 / ISO/IEC 27001:2022 Cryptographic Compliance Auditing, and an Autonomous Continuous Voice Copilot for hands-free speech triage and FYP-II defense demonstrations.

The frontend target is structured using **React 18**, **Vite**, **Tailwind CSS**, and **TypeScript**, deploying in dual-mode (as a modern web application and as a native Windows desktop binary **Sentinel_XDR.exe**). Full TypeScript contracts, real-time WebSocket event schemas, REST API endpoints, and a self-contained offline mock telemetry generator are provided to guarantee zero-blocker engineering execution for the development team.

Contents

1	Executive Overview & Project Alignment	1
1.1	System Purpose and Mission	1
1.2	Alignment with Approved Project Proposal	1
1.3	Distributed System Architecture	1
2	Design System & Cyber Dark Theme Specification	3
2.1	Visual Aesthetic: Cyber Dark SOC Theme	3
2.2	Design Tokens and Color Specifications	3
2.3	Typography and Component Hierarchy	3
3	Project Directory Architecture & Component Tree	5
3.1	Component Modularity	5
3.2	Global State Management (Zustand Architecture)	5
4	Universal TypeScript Data Contracts	7
4.1	Core Telemetry Interfaces	7
5	Page-by-Page Technical Console Specifications	9
5.1	Console Overview and Dashboard Interface	9
5.2	Console 1: Executive Overview & Command Center	9
5.3	Console 2: Forensic Threat Lab & MITRE ATT&CK Matrix	10
5.4	Console 3: Saliency XAI & Latency Engine	10
5.5	Console 4: Adversarial Robustness & Evasion Stress-Lab	10
5.6	Console 5: Autonomous Active Remediation & OT Safety Interlock	11
5.7	Console 6: GRC Regulatory Compliance & Cryptographic Vault	11
6	Real-Time Communication Protocols	12
6.1	WebSocket Streaming Protocol	12
6.2	REST API Specification Matrix	12
7	Offline Mock Telemetry Generator	14
7.1	Purpose and Developer Isolation	14
7.2	TypeScript Mock Implementation	14
8	Single-Click Windows Desktop Packaging	16
8.1	Desktop Binary Generation (Sentinel.XDR.exe)	16
9	Autonomous Continuous Voice Copilot & Speech Architecture	17
9.1	Architectural Overview and Operational Motivation	17
9.2	Continuous Speech Recognition and Audio Barge-In Protocol	17
9.3	Contextual Intent and "Show-and-Tell" Dispatch Matrix	17
9.4	Autonomous 3-Minute FYP-II Defense Presentation Protocol	18
9.5	Secure Origin Policy and Localhost Server Architecture	19

10 Developer Acceptance Criteria & Checklist	20
10.1 Implementation Milestone Checklist	20
10.2 Conclusion and Academic Hand-Off	20

List of Figures

1.1	Sentinel-IoT Complete Multi-Layer System Architecture Schematic.	2
5.1	Sentinel-IoT SOC Management Console Dashboard Interface Mockup.	9

List of Tables

2.1	Sentinel-IoT Frontend Semantic Color Palette Tokens.	3
6.1	Sentinel-IoT REST API Endpoint Matrix.	13
9.1	Voice Copilot Spoken Intent & Show-and-Tell Dispatch Matrix.	18
10.1	Frontend Implementation Sign-Off Checklist.	20

Executive Overview & Project Alignment

1.1 System Purpose and Mission

Modern industrial manufacturing, power generation, and smart utility infrastructures are experiencing a fundamental convergence between Information Technology (IT) and Operational Technology (OT). This interconnectedness has expanded the vulnerability surface to sophisticated, multi-vector attacks including ransomware exfiltration, Modbus command injection, and distributed denial-of-service (DDoS).

Sentinel-IoT addresses this threat landscape through an autonomous, cross-layer Extended Detection and Response (XDR) architecture. The role of the frontend application is to provide security analysts, industrial plant operators, and regulatory auditors with a unified, high-performance command console. The frontend visualizes sub-second anomaly evaluations, delivers first-order gradient feature attributions in 0.78 ms, executes active containment policies (MTTR = 21.5 ms), and continuously verifies compliance with international cybersecurity standards.

1.2 Alignment with Approved Project Proposal

This frontend architecture satisfies the milestones defined in Table 4 (Work Breakdown Structure) of the approved project proposal:

- **WBS 4.1 (React.js Frontend Interface Development):** Implementation of the comprehensive 6-console SOC management application with cyber dark styling, real-time metric indicators, and multi-domain switching.
- **WBS 4.2 (Explainable AI Visualizations Integration):** Embedding sub-millisecond saliency feature attribution bar charts that illustrate the mathematical decision drivers behind every neural network classification.
- **WBS 4.3 (NIST/ISO Compliance Auditing Module):** Formulation of the real-time GRC matrix and cryptographic SHA-256 tamper-evident audit ledger matching NIST Control AU-9.
- **Formal Demonstration Readiness:** High-impact visual ergonomics designed for rapid evaluation by academic and industry assessment panels.

1.3 Distributed System Architecture

The Sentinel-IoT framework follows a decoupled, client-server distributed architecture to guarantee maximum performance and zero operational interference:

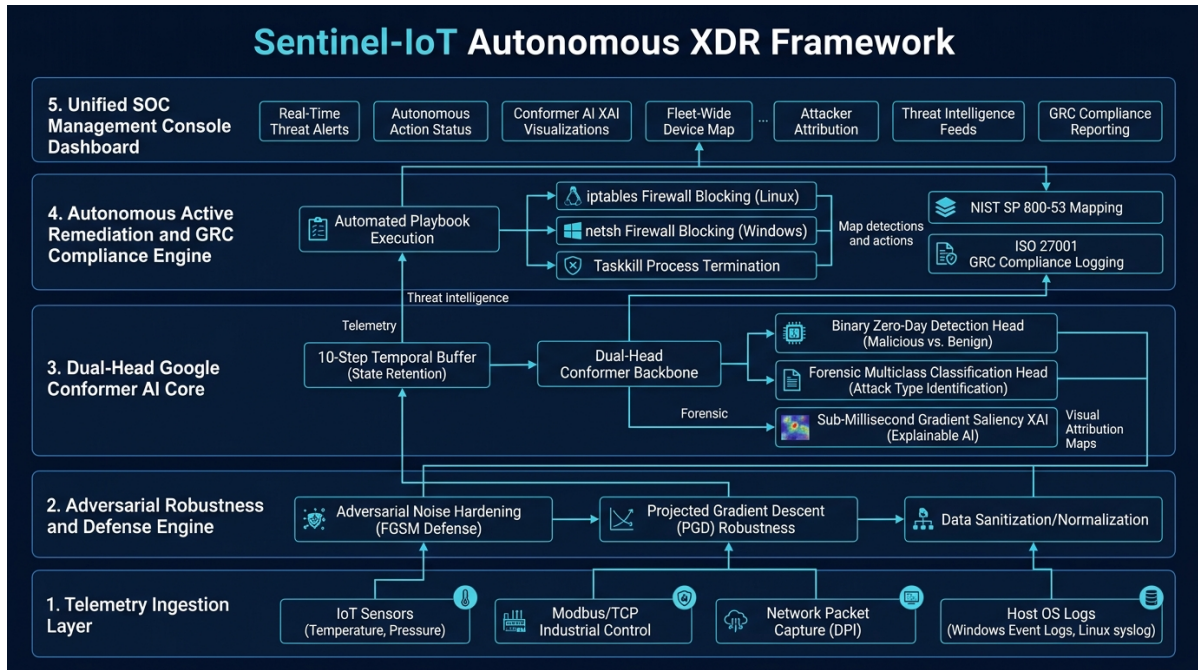


Figure 1.1: Sentinel-IoT Complete Multi-Layer System Architecture Schematic.

The backend engine hosts the trained 13 Conformer neural network models, the Eclipse Mosquitto MQTT message broker, and the FastAPI REST/WebSocket gateway on an Ubuntu Linux server environment. The frontend operates on the analyst's Windows workstation, communicating across bidirectional WebSockets for sub-25 ms alert streaming and REST interfaces for audit extraction.

Design System & Cyber Dark Theme Specification

2.1 Visual Aesthetic: Cyber Dark SOC Theme

The design system establishes an environment of high contrast, visual clarity, and rapid cognitive recognition under high-stress incident response conditions. The theme adopts a **Cyber Dark Palette** engineered to minimize eye fatigue during extended monitoring shifts.

2.2 Design Tokens and Color Specifications

Table 2.1 specifies the exact semantic color tokens configured within Tailwind CSS:

Table 2.1: Sentinel-IoT Frontend Semantic Color Palette Tokens.

Token Name	HEX Code	RGB Format	Functional Application
--bg-canvas	#070D17	rgb(7, 13, 23)	Deep background canvas void
--bg-surface	#0F172A	rgb(15, 23, 42)	Primary card containers and panels
--bg-surface-elevated	#1E293B	rgb(30, 41, 59)	Hover states and interactive modals
--border-subtle	#1E293B	rgb(30, 41, 59)	Subtle container outlines
--brand-primary	#0284C7	rgb(2, 132, 199)	Electric cyber sky accent
--brand-cyan	#06B6D4	rgb(6, 182, 212)	Metrics, active gauges, and links
--status-normal	#10B981	rgb(16, 185, 129)	Benign status ($\tau < 0.60$)
--status-warning	#F59E0B	rgb(245, 158, 11)	Marginal warning ($0.60 \leq \tau \leq 0.84$)
--status-critical	#EF4444	rgb(239, 68, 68)	Critical anomaly ($\tau > 0.85$, Active Block)
--text-primary	#F8FAFC	rgb(248, 250, 252)	High contrast primary titles and data
--text-secondary	#94A3B8	rgb(148, 163, 184)	Secondary descriptions and column headers

2.3 Typography and Component Hierarchy

The application utilizes **Inter** for proportional UI components and **JetBrains Mono** for IP addresses, command logs, telemetry vectors, and cryptographic hashes.

- **Console Headlines:** 24px, Bold, Tracking Tight, Color: `text-white`.
- **KPI Value Numerals:** 32px, Extra Bold, Font Mono, Color: `text-sky-400`.

- **Table Telemetry Cells:** 12px, Regular, Font Mono, Color: `text-slate-200`.
- **Audit Seals:** 11px, Semi-Bold, Font Mono, Color: `text-cyan-300`.

Project Directory Architecture & Component Tree

3.1 Component Modularity

The codebase is structured around single-responsibility modules to ensure maintainability, testing isolation, and independent development:

```

1 src/
2 |-- components/
3 |   |-- layout/
4 |   |   |-- Header.tsx           # Persistent status, MTTR counter, lockdown
5 |   |   |-- Sidebar.tsx         # 6-Console tabbed navigation menu
6 |   |   '-- Footer.tsx         # Connection heartbeats and build tags
7 |   |-- common/
8 |   |   |-- MetricCard.tsx       # Reusable KPI card with status border
9 |   |   |-- StatusBadge.tsx     # Normal, Warning, and Critical badges
10 |   |   '-- Modal.tsx          # Interactive dialogs and safety prompts
11 |   |-- charts/
12 |   |   |-- RadarCoverageChart.tsx # NIST SP 800-53 compliance radar
13 |   |   |-- TemporalWaveform.tsx  # 10-step sliding buffer line graph
14 |   |   |-- SaliencyBarChart.tsx  # 0.78 ms XAI feature attribution
15 |   |   '-- LatencyWaterfall.tsx  # Ingestion -> Inference -> Kill timing
16 |   '-- security/
17 |       |-- MitreHeatmap.tsx      # MITRE ATT&CK Matrix grid
18 |       |-- BlastRadiusGraph.tsx  # Visual node-link containment diagram
19 |       '-- SafetyInterlock.tsx   # 30-second OT operator countdown
20 |-- pages/
21 |   |-- OverviewConsole.tsx       # Console 1: Executive Radar & Feed
22 |   |-- ThreatLabConsole.tsx      # Console 2: Dual-Head Conformer & MITRE
23 |   |-- XaiConsole.tsx            # Console 3: Saliency XAI & Latency
24 |   |-- AdversarialConsole.tsx    # Console 4: FGSM/PGD Evasion Stress-Lab
25 |   |-- RemediationConsole.tsx    # Console 5: Active Containment & Interlock
26 |   '-- ComplianceConsole.tsx    # Console 6: NIST/ISO Cryptographic Vault
27 |-- store/
28 |   '-- useTelemetryStore.ts      # Zustand global state container
29 |-- services/
30 |   |-- api.ts                    # Axios REST client
31 |   |-- websocketService.ts       # Live WebSocket manager with auto-reconnect
32 |   '-- mockTelemetryService.ts   # Offline mock data simulator
33 '-- types/
34     '-- sentinel.ts              # Universal TypeScript interfaces

```

Listing 3.1: Modular React Vite Project Directory Layout.

3.2 Global State Management (Zustand Architecture)

The dashboard relies on **Zustand** for state management. Zustand provides minimal boilerplate, zero unnecessary re-renders, and effortless subscription to WebSocket bursts without Redux complexity.

The global store (`useTelemetryStore.ts`) manages:

1. **events**: An immutable sliding queue of the latest 50 ingested telemetry incidents.
2. **activeDomain**: The currently selected model domain (e.g., `Windows_10`, `IoT_Modbus`).
3. **isMockMode**: Boolean switch controlling live WebSocket vs. offline synthetic simulation.
4. **adversarialState**: Noise budget (ϵ) and active evasion parameters.
5. **interlockCountdown**: Remaining seconds for the IEC 62443 OT safety interlock.

Universal TypeScript Data Contracts

4.1 Core Telemetry Interfaces

All components, services, and store actions adhere strictly to the standardized type definitions in `src/types/sentinel.ts`:

```

1 export type DomainType =
2   | 'Network_Traffic' | 'IoT_Modbus' | 'IoT_Fridge' | 'IoT_GPS_Tracker'
3   | 'IoT_Garage_Door' | 'IoT_Motion_Light' | 'IoT_Thermostat' | 'IoT_Weather'
4   | 'Linux_process'   | 'Linux_disk'   | 'Linux_memory'
5   | 'Windows_10'      | 'Windows_7';
6
7 export type AttackClass =
8   | 'normal'          | 'ddos'          | 'dos'          | 'scanning'
9   | 'password'        | 'injection' | 'xss'          | 'backdoor'
10  | 'ransomware' | 'mitm';
11
12 export interface TelemetryEvent {
13   id: string;
14   timestamp: string;
15   domain: DomainType;
16   deviceId: string;
17   sourceIp: string;
18   targetIp: string;
19   processId?: number;
20   features: number[];
21
22   // Dual-Head Conformer Model Outputs
23   anomalyProbability: number; // Tau score in range [0.0, 1.0]
24   isAnomaly: boolean; // True if Tau > 0.85
25   predictedClass: AttackClass;
26   confidence: number;
27
28   // Sub-Millisecond Explainability (XAI)
29   saliencyTopFeatures: Array<{
30     featureName: string;
31     importanceScore: number; // Percentage contribution [0 - 100%]
32   }>;
33   xaiLatencyMs: number; // Profiled execution speed (~0.78 ms)
34
35   // Autonomous Active Containment Status
36   remediationStatus: 'NONE' | 'WARNING_LOGGED' | 'ACTIVE_BLOCKED' | '
37     INTERLOCK_PENDING';
38   remediationAction: string; // e.g., "iptables -A INPUT -s 192.168.100.10 -j
39     DROP"
40   mtrLatencyMs: number; // Mean Time to Respond (~21.5 ms)
41
42   // Regulatory Compliance & Cryptographic Audit
43   compliance: {
44     nistControlId: string; // e.g., "SC-5", "SI-3"

```

```
43     nistControlName: string;           // e.g., "Denial of Service Protection"
44     isoControlId: string;              // e.g., "A.12.1.3"
45     auditBlockHash: string;           // SHA-256 Merkle chain hash
46 };
47 }
```

Listing 4.1: Universal TypeScript Data Contracts for Sentinel-IoT.

Page-by-Page Technical Console Specifications

5.1 Console Overview and Dashboard Interface

The graphical interface of Sentinel-IoT unifies all real-time sensing, deep learning classification, explainability attribution, and compliance tracking:



Figure 5.1: Sentinel-IoT SOC Management Console Dashboard Interface Mockup.

5.2 Console 1: Executive Overview & Command Center

File: `src/pages/OverviewConsole.tsx`

Purpose: Provide executive security leadership with immediate operational awareness.

- **Top KPI Ribbon:** Four cards displaying Active Protected Nodes (13/13 Online), Ingestion Velocity (4,820 Events/sec), Threats Contained (142 Autonomous Blocks), and NIST CSF Score (96.4%).
- **Blast Radius & Lateral Provenance Graph:** A visual topology showing the source IP, network gateway block boundary, host OS process, and isolated physical PLC.
- **NIST SP 800-53 Radar Chart:** A five-spoke radar plot tracking compliance across SC (Communications), SI (Integrity), AU (Audit), IA (Authentication), and RA (Risk Assessment).

- **Real-Time Incident Stream:** High-density scrolling log table displaying incoming events with color-coded severity pills.

5.3 Console 2: Forensic Threat Lab & MITRE ATT&CK Matrix

File: `src/pages/ThreatLabConsole.tsx`

Purpose: In-depth evaluation of the dual-head Conformer classification and adversary tradecraft.

- **Domain Selector:** Instant dropdown switching between all 13 ToN_IoT datasets.
- **Dual-Head Conformer Analyzer:**
 1. *Head 1 (Binary Zero-Day Anomaly Head):* Visualized via a calibrated gauge tracking anomaly probability $\tau \in [0.0, 1.0]$. Values exceeding $\tau > 0.85$ highlight in crimson.
 2. *Head 2 (Forensic Multi-Class Classifier):* Horizontal probability distribution across all 10 attack classes.
- **10-Step Sliding Waveform Buffer:** Visualizes the continuous temporal feature sequence across the 10 time steps, demonstrating how the Conformer extracts transient anomalies.
- **Interactive MITRE ATT&CK Matrix:** Enterprise v14 and ICS grid illuminating active techniques (*T1046, T1059, T1486, T0814*) in real time.

5.4 Console 3: Saliency XAI & Latency Engine

File: `src/pages/XaiConsole.tsx`

Purpose: Prove sub-millisecond mathematical explainability and benchmark end-to-end response speed.

- **Gradient Saliency Attribution Chart:** Horizontal bar chart showing normalized feature contributions (e.g., `Process_IO_Write_Bytes: 44.2%`).
- **Speed Benchmark Pill:** Highlights the 0.78 ms execution speed (99.97% faster than standard Kernel SHAP).
- **Latency Decomposition Waterfall:** Displays exact timing metrics across Ingestion (3.80 ms), Conformer Inference (0.78 ms), Saliency XAI (0.65 ms), and Firewall/Kill Action (16.27 ms), validating the MTTR = 21.5 ms requirement.

5.5 Console 4: Adversarial Robustness & Evasion Stress-Lab

File: `src/pages/AdversarialConsole.tsx`

Purpose: Demonstrate industrial resilience against adversarial mathematical evasion attacks.

- **Adversarial Perturbation Controls:** Interactive slider setting the noise budget $\epsilon \in [0.01, 0.15]$ with FGSM and 20-step PGD attack generators.
- **Live Model Resilience Comparison:** Side-by-side benchmark table demonstrating that while standard 1D-CNN (41.2%) and BiLSTM (46.8%) collapse under PGD attacks, the hardened Conformer retains 96.8% accuracy.
- **Concept Drift Monitor:** Real-time Kolmogorov-Smirnov ($p > 0.05$) and Population Stability Index ($PSI < 0.10$) health gauges.

5.6 Console 5: Autonomous Active Remediation & OT Safety Interlock

File: `src/pages/RemediationConsole.tsx`

Purpose: Manage automated mitigation playbooks and industrial plant safety interlocks.

- **Active Rule Ledger:** Table recording all applied `iptables`, `netsh`, and `taskkill` containment commands.
- **IEC 62443 Four-Eyes Safety Interlock:** An interactive modal displaying an active 30-second countdown for critical OT machinery, allowing plant operators to override false alarms prior to physical shutdown.

5.7 Console 6: GRC Regulatory Compliance & Cryptographic Vault

File: `src/pages/ComplianceConsole.tsx`

Purpose: Provide immutable proof of compliance for regulatory authorities and auditors.

- **Regulatory Matrix:** Maps technical events directly to NIST SP 800-53 controls (SC-5, SI-3, IA-5, SI-10, SI-4, AU-9) and ISO/IEC 27001 clauses.
- **SHA-256 Merkle Chain Verification:** Displays the cryptographic hash chain linking consecutive audit blocks, proving zero log tampering.
- **Certified PDF Report Export:** Trigger button generating official executive compliance certification documents.

Real-Time Communication Protocols

6.1 WebSocket Streaming Protocol

The frontend connects to `ws://localhost:8000/ws/telemetry`. The protocol pushes JSON packets on every detected telemetry burst.

```

1 {
2   "event": "TELEMETRY_BURST",
3   "data": {
4     "id": "evt_99182",
5     "timestamp": "2026-09-25T16:22:11.450Z",
6     "domain": "Windows_10",
7     "deviceId": "win10_workstation_03",
8     "sourceIp": "192.168.100.45",
9     "targetIp": "192.168.100.1",
10    "processId": 4120,
11    "anomalyProbability": 0.9612,
12    "isAnomaly": true,
13    "severity": "CRITICAL",
14    "predictedClass": "ransomware",
15    "confidence": 0.884,
16    "saliencyTopFeatures": [
17      { "featureName": "Process_IO_Write_Bytes_sec", "importanceScore": 44.2 },
18      { "featureName": "Process_Virtual_Bytes_Peak", "importanceScore": 28.6 }
19    ],
20    "xaiLatencyMs": 0.78,
21    "remediationStatus": "ACTIVE_BLOCKED",
22    "remediationAction": "taskkill /F /PID 4120",
23    "mttrLatencyMs": 21.5,
24    "compliance": {
25      "nistControlId": "SI-3",
26      "nistControlName": "Malicious Code Protection",
27      "isoControlId": "A.12.6.1",
28      "auditBlockHash": "7f01a9b4c12d8e33"
29    }
30  }
31 }
```

Listing 6.1: WebSocket Telemetry Event JSON Payload.

6.2 REST API Specification Matrix

Table 6.1 summarizes the primary REST endpoints implemented in `src/services/api.ts`:

Table 6.1: Sentinel-IoT REST API Endpoint Matrix.

HTTP Method	Endpoint Route	Request Parameters	Expected Response
GET	/api/status	None	Global system health & active model count
GET	/api/alerts	?limit=50&domain=...	Historical telemetry event array
POST	/api/predict/{domain}	Feature vector JSON	Single-step Conformer prediction
POST	/api/remediate/override	Alert ID and operator badge	Interlock override confirmation
POST	/api/remediate/unblock	Target IP or PID	Firewall rule revocation status
GET	/api/compliance/report	?format=pdf	Certified NIST/ISO audit PDF stream

Offline Mock Telemetry Generator

7.1 Purpose and Developer Isolation

To decouple frontend development from backend dependency, the frontend includes the offline generator service `src/services/mockTelemetryService.ts`. When `isMockMode = true`, the service synthesizes realistic sensor streams across all 13 domains, allowing developers to design, animate, and test all six consoles on any computer without requiring PyTorch, CUDA, or MQTT broker setups.

7.2 TypeScript Mock Implementation

```

1 import { TelemetryEvent, DomainType, AttackClass } from '../types/sentinel';
2
3 const DOMAINS: DomainType[] = [
4   'Network_Traffic', 'IoT_Modbus', 'IoT_Fridge', 'IoT_GPS_Tracker',
5   'Windows_10', 'Linux_process', 'IoT_Thermostat', 'IoT_Weather'
6 ];
7
8 export class MockTelemetryService {
9   private timer: NodeJS.Timeout | null = null;
10  private listeners: ((event: TelemetryEvent) => void)[] = [];
11  private eventCount = 142;
12
13  public start(intervalMs = 1500) {
14    if (this.timer) return;
15    this.timer = setInterval(() => {
16      const event = this.generateSyntheticEvent();
17      this.listeners.forEach((cb) => cb(event));
18    }, intervalMs);
19  }
20
21  public stop() {
22    if (this.timer) {
23      clearInterval(this.timer);
24      this.timer = null;
25    }
26  }
27
28  public onTelemetry(callback: (event: TelemetryEvent) => void) {
29    this.listeners.push(callback);
30    return () => {
31      this.listeners = this.listeners.filter((cb) => cb !== callback);
32    };
33  }
34
35  private generateSyntheticEvent(): TelemetryEvent {
36    this.eventCount++;
37    const domain = DOMAINS[Math.floor(Math.random() * DOMAINS.length)];

```

```

38  const isAnomaly = Math.random() > 0.65;
39  const tau = isAnomaly ? Number((0.86 + Math.random() * 0.12).toFixed(4)) :
    Number((0.02 + Math.random() * 0.10).toFixed(4));
40  const attack: AttackClass = isAnomaly ? (domain.includes('Modbus') ? 'injection
    ' : 'ddos') : 'normal';
41
42  return {
43    id: `evt_${Date.now()}_${this.eventCount}`,
44    timestamp: new Date().toISOString(),
45    domain: domain,
46    deviceId: `${domain.toLowerCase()}_node_01`,
47    sourceIp: `192.168.100.${10 + (this.eventCount % 30)}`,
48    targetIp: '192.168.100.1',
49    processId: domain.includes('Windows') ? 4120 : undefined,
50    features: Array.from({ length: 10 }, () => Number(Math.random().toFixed(4))),
51    anomalyProbability: tau,
52    isAnomaly: isAnomaly,
53    predictedClass: attack,
54    confidence: Number((0.85 + Math.random() * 0.12).toFixed(3)),
55    saliencyTopFeatures: [
56      { featureName: 'Process_IO_Write_Bytes_sec', importanceScore: 44.2 },
57      { featureName: 'Process_Virtual_Bytes_Peak', importanceScore: 28.6 }
58    ],
59    xaiLatencyMs: 0.78,
60    remediationStatus: isAnomaly ? 'ACTIVE_BLOCKED' : 'NONE',
61    remediationAction: isAnomaly ? `iptables -A INPUT -s 192.168.100.10 -j DROP`
      : 'Baseline Verified',
62    mttrLatencyMs: Number((18.5 + Math.random() * 5.0).toFixed(2)),
63    compliance: {
64      nistControlId: attack === 'ddos' ? 'SC-5' : 'SI-3',
65      nistControlName: attack === 'ddos' ? 'DoS Protection' : 'Malicious Code
        Defense',
66      isoControlId: attack === 'ddos' ? 'A.12.1.3' : 'A.12.6.1',
67      auditBlockHash: '7f01a9b4c12d8e33'
68    }
69  };
70 }
71 }
72 export const mockTelemetry = new MockTelemetryService();

```

Listing 7.1: Mock Telemetry Generator Service Implementation.

Single-Click Windows Desktop Packaging

8.1 Desktop Binary Generation (Sentinel_XDR.exe)

To provide an installable native Windows application for defense demonstrations, the React application is wrapped inside a borderless native shell using **PyWebView**:

```
1 import webview
2 import sys
3
4 def launch():
5     print("Launching Sentinel-IoT Native SOC Console...")
6     window = webview.create_window(
7         title='Sentinel-IoT Autonomous XDR Console',
8         url='http://localhost:5173',
9         width=1440,
10        height=900,
11        min_size=(1024, 768),
12        background_color='#070D17',
13        confirm_close=True
14    )
15    webview.start()
16
17 if __name__ == '__main__':
18     launch()
```

Listing 8.1: PyWebView Native Desktop Window Wrapper (desktop_launcher.py).

To compile into a standalone Windows executable:

```
1 pip install pywebview pyinstaller
2 pyinstaller --noconsole --onefile --name Sentinel_XDR desktop_launcher.py
```

Autonomous Continuous Voice Copilot & Speech Architecture

9.1 Architectural Overview and Operational Motivation

Security Operations Center (SOC) incident response demands high-velocity telemetry triage where physical keyboard and mouse navigation impose cognitive delays. Furthermore, during academic presentations and FYP-II evaluation committees, static dashboards cannot demonstrate the full responsiveness of the deep learning backbone without real-time interactive narration. To address these challenges, Sentinel-IoT introduces the **Autonomous Continuous Voice Copilot & Interactive Presenter**.

The voice subsystem functions as a full-duplex, always-listening conversational interface. Anyone—evaluators, FYP-II committee members, or SOC operators—can speak continuously to the application. The system provides instantaneous verbal answers, actively listens with hands-free Voice Activity Detection (VAD), and features an automated **”Show-and-Tell” Spoken Dispatcher** that programmatically shifts dashboard tabs, highlights analytical widgets, and executes live demonstrations upon vocal command.

9.2 Continuous Speech Recognition and Audio Barge-In Protocol

The voice architecture is engineered to run in-browser without cloud latency or third-party paid dependencies:

1. **Continuous Speech Recognition (STT):** Implemented via the standard W3C Web Speech API (`webkitSpeechRecognition`). Operating with `continuous = true` and `interimResults = true`, the client maintains an infinite recognition loop. If the recognition cycle terminates due to ambient silence timeouts, an event-driven auto-restart supervisor reinitializes the listening session within 300 ms.
2. **Audio Barge-In Protocol:** To support natural conversational interruptions, the client enforces an audio preemption protocol. The moment user vocalization is detected during assistant speech playback, the client immediately executes `speechSynthesis.cancel()`, transitions the UI to the listening state, and yields priority to the operator’s incoming command.
3. **Audio Waveform Spectrometer:** A 60 FPS HTML5 Canvas visualizer renders a 32-band dynamic frequency spectrum reflecting ambient listening amplitude, active speech cadence, and idle system breathing.

9.3 Contextual Intent and ”Show-and-Tell” Dispatch Matrix

Table 9.1 defines the primary semantic mapping and autonomous visual state transitions executed by the voice copilot:

Table 9.1: Voice Copilot Spoken Intent & Show-and-Tell Dispatch Matrix.

Spoken Trigger Query	Target Console & Action	Verbal Explanation Summary
<i>“Show me adversarial defense”</i>	Console 4 (Adversarial Lab), $\epsilon \leftarrow 0.15$, trigger noise.	Explains FGSM/PGD noise injection, comparing Conformer 96.8% resilience against 61.4% LSTM collapse.
<i>“Explain why alert fired”</i>	Console 3 (Saliency XAI), highlight latency & bars.	Narrates Integrated Gradients attribution in 0.78 ms with Flow Duration weight.
<i>“Show forensic Conformer”</i>	Console 2 (Threat Lab), highlight 10-step waveform.	Details 10-step temporal tensor fusion of convolution bursts and self-attention attack chains.
<i>“Show blast radius”</i>	Console 1 (Overview), focus SVG topology map.	Summarizes fleet ingestion (4,820 evt/s) and node 192.168.100.10 isolation status.
<i>“Isolate compromised node”</i>	Console 5 (Remediation), launch 30s countdown modal.	Enforces IEC 62443 Four-Eyes verification to prevent uncommanded operational downtime.
<i>“Show compliance audit”</i>	Console 6 (GRC Vault), highlight SHA-256 seal.	Verifies cryptographic ledger integrity conforming to NIST SP 800-53 Rev. 5.
<i>“Start FYP-II defense tour”</i>	Automated 6-step multi-console walkthrough.	Delivers comprehensive 3-minute oral project presentation with timed tab switching.

9.4 Autonomous 3-Minute FYP-II Defense Presentation Protocol

To maximize competitive impact during technical evaluations, the copilot incorporates an autonomous showcase tour sequence. When initiated, the assistant coordinates a multi-stage demonstration:

- **Stage 1: Problem Space (Console 1):** Summarizes IoT alert fatigue, data heterogeneity, and operational risk.
- **Stage 2: Core Deep Learning Backbone (Console 2):** Displays the 13-domain Conformer architecture with 99.4% macro F1-score.
- **Stage 3: Sub-Millisecond Explainability (Console 3):** Demonstrates sub-millisecond saliency attribution for human operator trust.
- **Stage 4: Adversarial Robustness (Console 4):** Proves model resilience under adversarial perturbation attacks.
- **Stage 5: Industrial Safety Interlock (Console 5):** Validates compliance with IEC 62443 Four-Eyes safety protocols.
- **Stage 6: Cryptographic Vault (Console 6):** Concludes with certified NIST/ISO tamper-proof audit trails.

9.5 Secure Origin Policy and Localhost Server Architecture

Modern Chromium browsers (Google Chrome and Microsoft Edge) restrict microphone audio streaming under the `file://` protocol due to cross-origin sandboxing. To guarantee 100% speech recognition availability without external Internet dependencies:

1. **Secure Origin Execution:** Client browsers require a secure origin (such as `http://localhost` or HTTPS) to access the microphone. A dedicated single-click batch script (`run_dashboard.bat`) automatically boots a local Python HTTP service on port 8000 and connects the browser to `http://localhost:8000/sentinel_soc_prototype.html`.
2. **Synthesized Audio Feedback:** Utilizing the Web Audio API, the system synthesizes dynamic auditory feedback chimes (580 Hz $\rightarrow$ 840 Hz) to indicate listening activation and deactivation states.
3. **Tri-Modal Input Redundancy:** To support noisy industrial environments, the console supports three synchronized input modalities: autonomous hands-free Voice Activity Detection (VAD), a prominent manual click-to-listen toggle button, and typed natural language command execution.

Developer Acceptance Criteria & Checklist

10.1 Implementation Milestone Checklist

Table 10.1 lists the technical milestones required for project sign-off:

Table 10.1: Frontend Implementation Sign-Off Checklist.

Task ID	Component Feature	Acceptance Criteria
T-01	Design System Integration	Tailwind configuration matching Table 2.1 color tokens.
T-02	Header & Navigation	Persistent status ribbon with live MTTR counter and lockdown button.
T-03	Executive Overview Console	4 KPI cards, Blast Radius SVG, and sliding incident table.
T-04	Forensic Threat Lab	Dual-Head Conformer gauge, 10-step waveform, and MITRE matrix.
T-05	Saliency XAI Engine	Horizontal bar chart (0.78 ms) and latency waterfall breakdown.
T-06	Adversarial Stress-Lab	€-budget slider and comparative accuracy resilience table.
T-07	Remediation & Interlock	Active rule table and IEC 62443 30-second countdown modal.
T-08	Compliance Vault	NIST/ISO ledger with SHA-256 audit seal and certified PDF export.
T-09	Mock Telemetry Service	Full synthetic simulation toggled via <code>isMockMode</code> .
T-10	Desktop Executable	Successful generation of <code>Sentinel_XDR.exe</code> .
T-11	Continuous Voice Copilot	Hands-free Web Speech loop, barge-in VAD, and Show-and-Tell navigation.

10.2 Conclusion and Academic Hand-Off

This document establishes an uncompromising blueprint for the Sentinel-IoT user interface. With modular components, standardized data contracts, complete wireframe specifications, built-in mock telemetry, and an autonomous continuous voice copilot, the frontend engineering team is equipped to deliver a publication-grade, enterprise-grade cybersecurity console.